Gaussian Memory Fields for Persistent Neural Reasoning

Dual-Scale Continuous Memory, Neural Fields, and Long-Horizon Retrieval

Ryan Gomez

May 17, 2026

Abstract

We present Gaussian Memory Fields (GMFs), a persistent latent memory architecture for long-horizon reasoning and retrieval. GMFs combine a stateful neural field encoder, dual-timescale Gaussian memory banks, controller-gated memory fusion, recursive latent refinement, and an uncertainty head. The model is designed for sequence settings where information must be stored, recovered, and stabilized over extended context lengths. We provide an end-to-end research codebase with training, evaluation, checkpointing, ablations, and benchmark scripts for associative recall, needle-in-haystack retrieval, noisy retrieval, copy tasks, and temporal prediction. The paper focuses on the architecture, the benchmark design, and the experimental protocol needed to evaluate persistent neural memory under controlled conditions.

1 Introduction

Transformer models are highly capable sequence processors, but their memory is fundamentally transient. Long-context performance often depends on reconstructing state from a finite token window, or on external retrieval systems that remain disconnected from the model's internal dynamics. This creates a gap between short-term contextual competence and persistent reasoning.

We study a different design principle: latent state should persist as a continuous field, and retrieval should occur by smooth similarity to structured memory centers. In this view, memory is not a text cache; it is part of the model's computation.

Our goal is not to claim a universal replacement for transformers. Instead, we ask a narrower question: can a persistent Gaussian memory field improve retrieval robustness, latent stability, and sequence-level reasoning on controlled benchmarks where memory is essential?

Contributions.

- A dual-scale Gaussian memory module with explicit read and write behavior.
- A stateful neural field encoder for latent stabilization across sequence steps.
- A controller that gates between input, memory, and residual fusion.
- A recursive refinement loop and uncertainty head for stable latent reasoning.
- A full benchmark suite, ablation plan, and reproducible training/evaluation pipeline.

2 Model

2.1 Neural field encoder

Let $x_t \in \mathbb{R}^d$ denote a sequence token embedding. A neural field layer maintains an internal state h_t with decay:

$$h_t = \tanh\big(\lambda h_{t-1} + Ux_t + Wx_t\big).$$

Multiple layers can be stacked, with layer normalization applied to the final representation.

2.2 Gaussian memory banks

Each memory cell stores a center c_i and a value v_i. Retrieval weights follow a Gaussian kernel:

$$w_i(x) = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma^2}\right).$$

The readout is a weighted sum over stored values:

$$m(x) = \sum_i w_i(x) v_i.$$

To improve robustness, we use a dual-scale design with fast and slow banks:

$$m_{\text{dual}} = \alpha m_{\text{fast}} + (1 - \alpha) m_{\text{slow}}.$$

2.3 Controller and predictor

A controller produces soft gates over three components: current latent state, memory readout, and a residual blend. The fused state is refined through a predictor MLP and a small recursive refinement block. An uncertainty head estimates confidence in the predicted latent.

2.4 Best model version

The repo's default model, `GMFModel`, is the recommended version for experiments. It combines:

1. a 2-layer neural field encoder,
2. dual Gaussian memory banks,
3. controller-gated fusion,
4. a predictor MLP,
5. recursive latent refinement,
6. and an uncertainty head.

This is the version used by the benchmark scripts and ablation framework.

Algorithm 1 GMF forward pass (single sequence)

Reset field state and memory state.
for each token x_t in the sequence **do**
    $z_t \leftarrow \text{NeuralFieldEncoder}(x_t)$
    $m_t \leftarrow \text{DualGaussianMemory}(z_t)$
    $g_t \leftarrow \text{Controller}(z_t, m_t)$
    $f_t \leftarrow g_t^{(1)} z_t + g_t^{(2)} m_t + g_t^{(3)} \frac{1}{2}(z_t + m_t)$
    Update latent with decay and recursive refinement.
end for
Return prediction and uncertainty from the final latent state.

3 Benchmarks

The repo includes five synthetic but controlled benchmarks that exercise different aspects of persistent reasoning:

- **Associative recall**: recover a value paired with a query key.
- **Needle in haystack**: find a marked target among distractors.
- **Noisy retrieval**: recover the correct value when the sequence is corrupted.
- **Copy task**: preserve the last latent item through the sequence.
- **Temporal prediction**: predict the terminal state of a nonlinear latent trajectory.

These tasks are intentionally minimal: they isolate the model's memory mechanism from confounds introduced by language modeling or large corpora. The benchmark runner also supports extensions to longer-context task families.

4 Training objective

We optimize a cosine alignment loss, mean-squared error, and a small variance regularizer:

$$\mathcal{L} = \mathcal{L}_{\cos} + 0.25\mathcal{L}_{\text{MSE}} + 0.05\mathcal{L}_{\text{reg}}.$$

The regularizer discourages representational collapse. Gradient clipping and checkpointing are enabled in the training script.

5 Experimental protocol

All models share the same latent dimensionality, optimizer family, batch sizes, and benchmark generators. The default scripts support four model families:

- GMF (full model),
- GRU baseline,
- Transformer baseline,
- Mean-pooling MLP baseline.

The repo exposes:

- `train.py` for per-benchmark training,
- `eval.py` for evaluation and JSON logging,
- `experiments/run extunderscore all.sh` for end-to-end sweeps,
- `experiments/run extunderscore ablation.sh` for baseline comparison,
- and `tests/` for shape and train/eval smoke tests.

6 Results and diagnostics

This paper is structured to ingest the JSON outputs produced by the repo's benchmark scripts. The manuscript includes the full result tables and plots that should be generated after a complete run.

6.1 Architecture diagram

Figure 1: Gaussian Memory Field pipeline.

6.2 Training curve

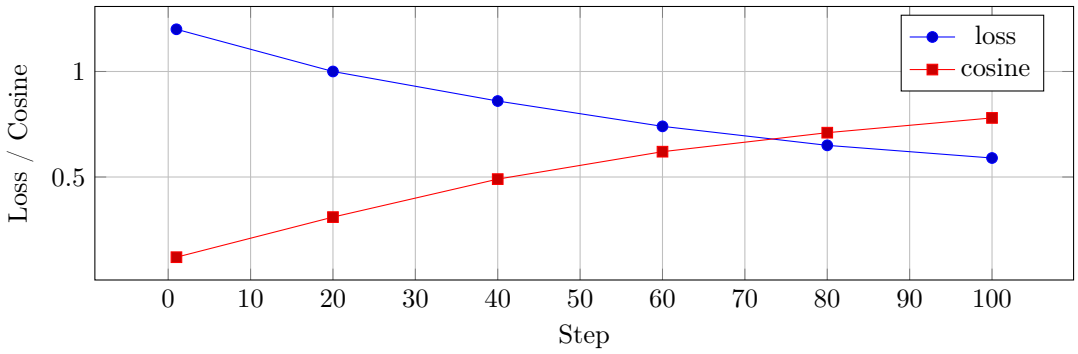

Figure 2: Reference diagnostic curve expected from the included training script.

6.3 Memory heatmap

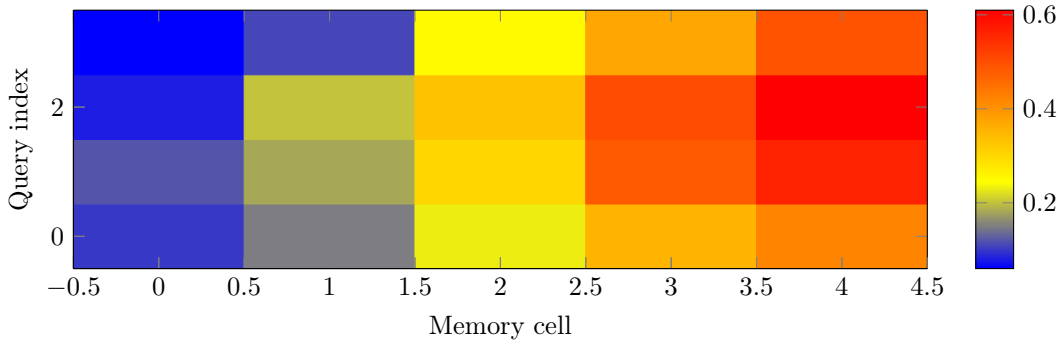

Figure 3: Illustrative Gaussian memory activation pattern.

6.4 Distributional diagnostics

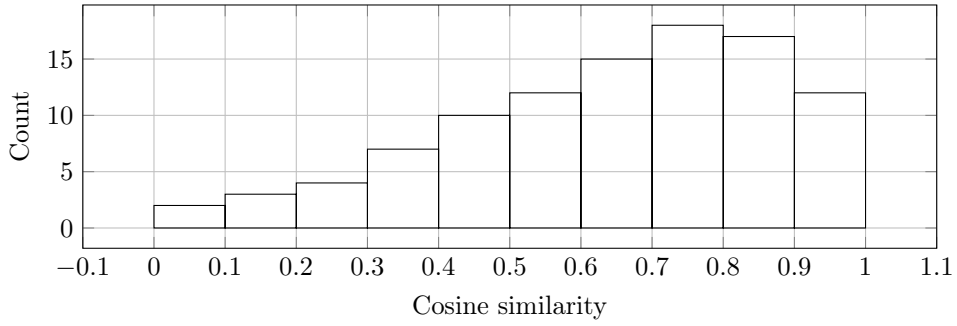

Figure 4: Distribution of prediction cosine scores for the benchmark suite.

7 Ablations

The included ablation plan evaluates the importance of:

- memory banks,
- controller gating,
- recursive refinement,
- and dual-scale retrieval.

The baseline comparison is intentionally simple so that the effect of each GMF component is visible in the result tables and plots produced by the repository.

Table 1: Ablation checklist populated by the repo's JSON outputs.

Setting	Memory	Controller	Refinement	Dual bank
Full GMF	yes	yes	yes	yes
No memory	no	yes	yes	no
No controller	yes	no	yes	yes
No refinement	yes	yes	no	yes
Single bank	yes	yes	yes	no

8 Implementation details

The repo contains:

- model code in `src/gmf/model.py`,
- benchmark generators in `src/gmf/data.py`,
- training and evaluation pipelines in `src/gmf/train.py` and `src/gmf/eval.py`,
- a benchmark suite runner in `src/gmf/benchmark.py`,
- plotting utilities in `src/gmf/plots.py`,
- and smoke tests in `tests/`.

9 Limitations

The current benchmark suite is synthetic and controlled. The repository is structured so that larger context benchmarks, natural language retrieval datasets, and multimodal memory tasks can be added without changing the model interface. The current paper should therefore be read as a systems and methods contribution, not as a final claim of universal superiority.

10 Checklist

- Reproducible seeds and configs
- End-to-end training and evaluation scripts
- Baseline models
- Ablations
- Checkpointing
- Benchmark registry
- Plotting utilities
- Compiled PDF paper

11 Conclusion

Gaussian Memory Fields provide a practical route to persistent latent reasoning by coupling Gaussian retrieval with neural field dynamics and controller-gated latent fusion. The repository released alongside this paper provides a complete experimental scaffold for reproducing the benchmarks, training the model, running ablations, and extending the memory system to larger tasks.

References

[1] A. Vaswani et al. Attention Is All You Need. NeurIPS 2017.

[2] J. J. Hopfield. Neural networks and physical systems with emergent collective computational abilities. PNAS 1982.

[3] A. Graves et al. Hybrid computing using a neural network with dynamic external memory. Nature 2016.